

Desenvolvimento Back-end II

- **Disciplina:** Desenvolvimento Back-end II
- **Prof.** Rossini Pena Abrantes
- rossini.abrantes@ifsudestemg.edu.br



INSTITUTO FEDERAL
Sudeste de Minas Gerais

Sumário

- Recursos HTTP
 - Introdução
 - Métodos
 - Códigos

Sumário

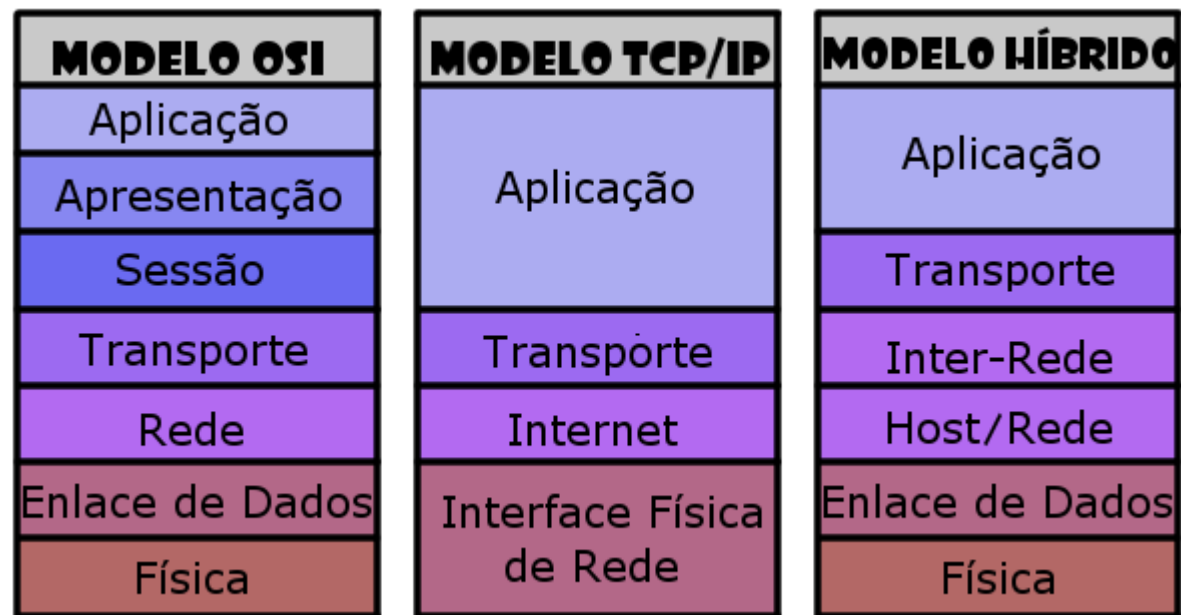
- Recursos HTTP
 - **Introdução**
 - Métodos
 - Códigos

Introdução

- HTTP é um **protocolo** de Redes de Computadores
- Um protocolo é uma **regra** de comunicação
 - Ela **permite** que 2 partes se comuniquem utilizando a mesma regra de comunicação
- Uma rede de computador e internet funciona graças a uma **pilha** de protocolos

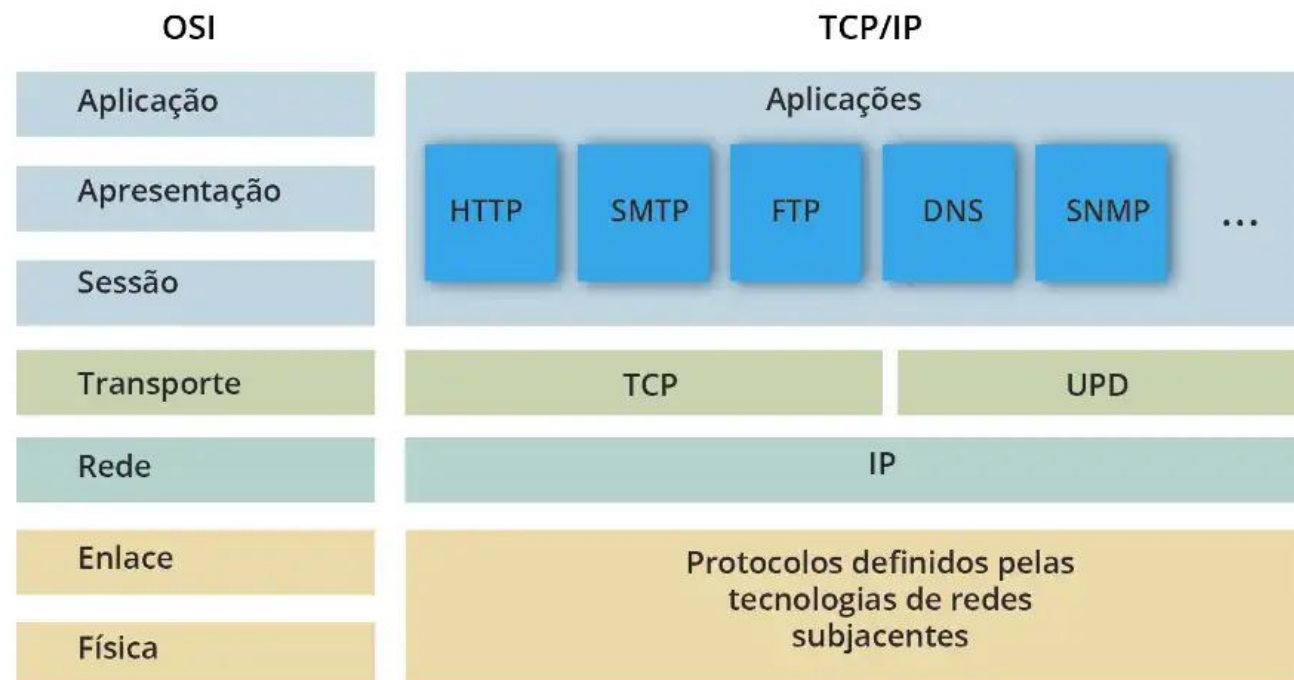
Introdução

- Pilhas de Protocolos



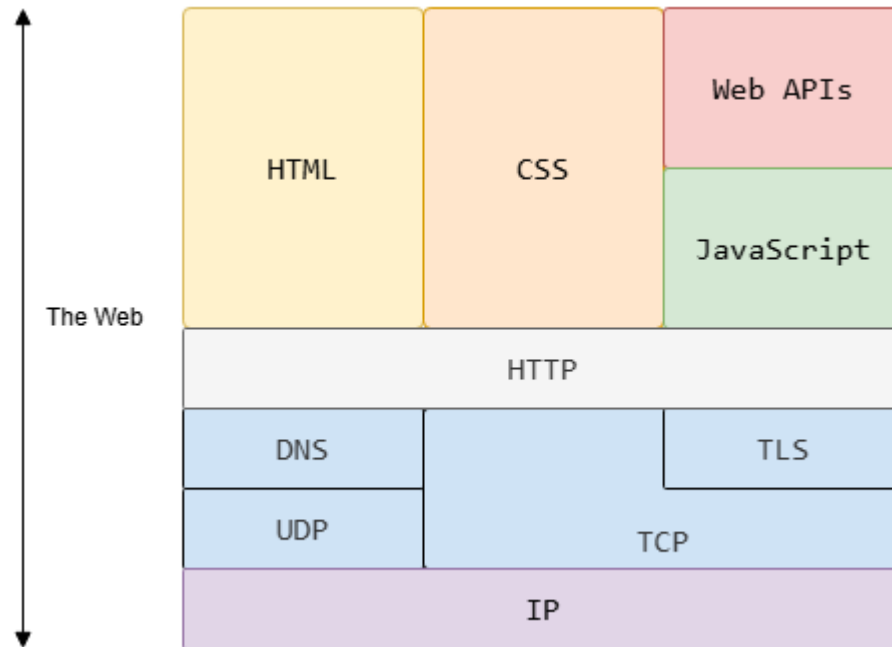
Introdução

- HTTP é um protocolo de Aplicação, na pilha de protocolos



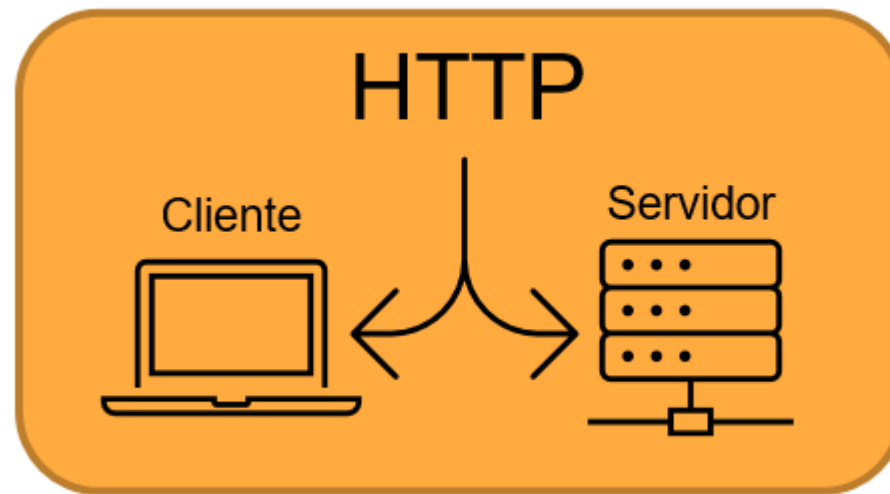
Introdução

- HTTP é quem possibilita a **transferência** de arquivos HTML, CSS, JS de **aplicações web**



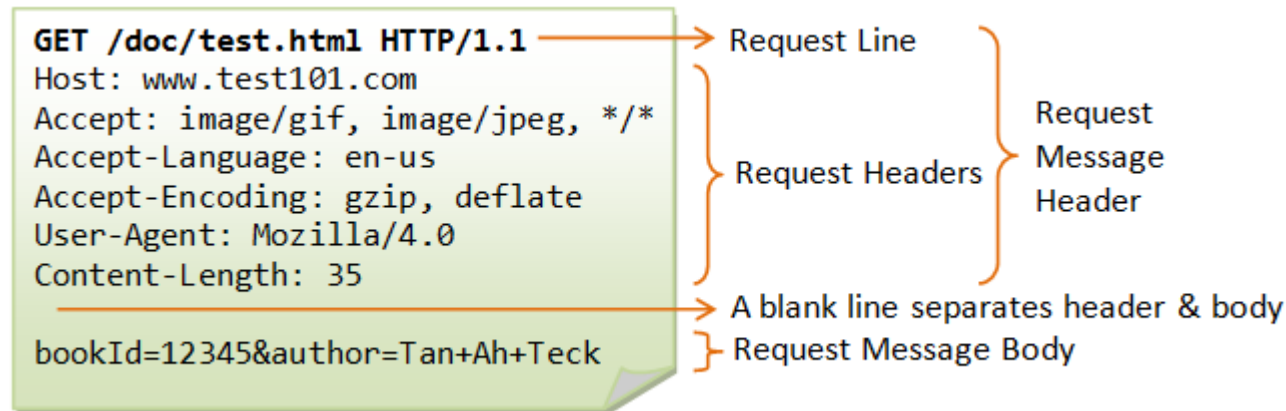
Introdução

- HTTP é a "**linguagem**" de comunicação entre Navegadores Web e Aplicações Web



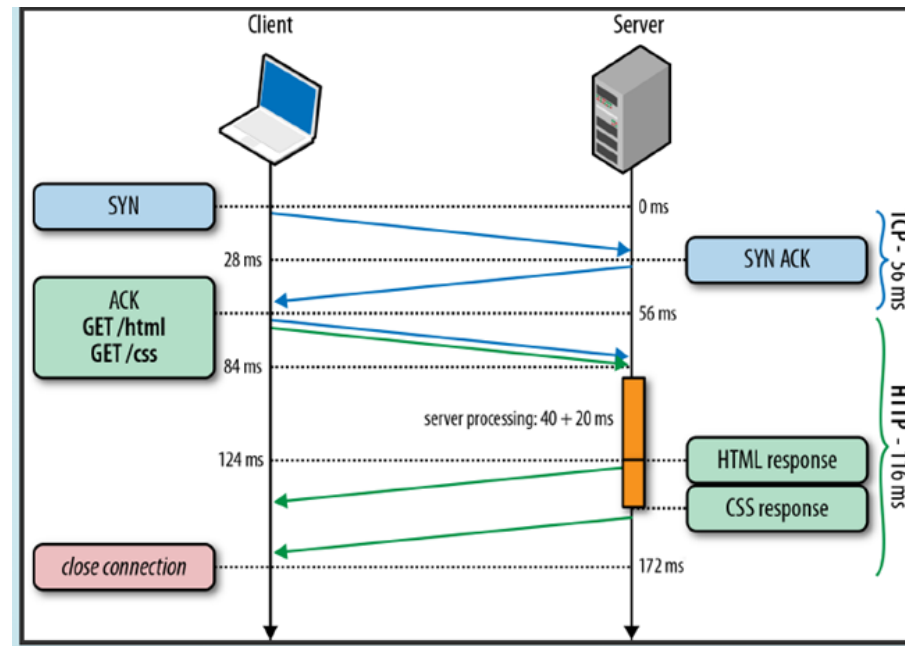
Introdução

- Um protocolo defini **2 regras** principais:
 - **Formato** das mensagens



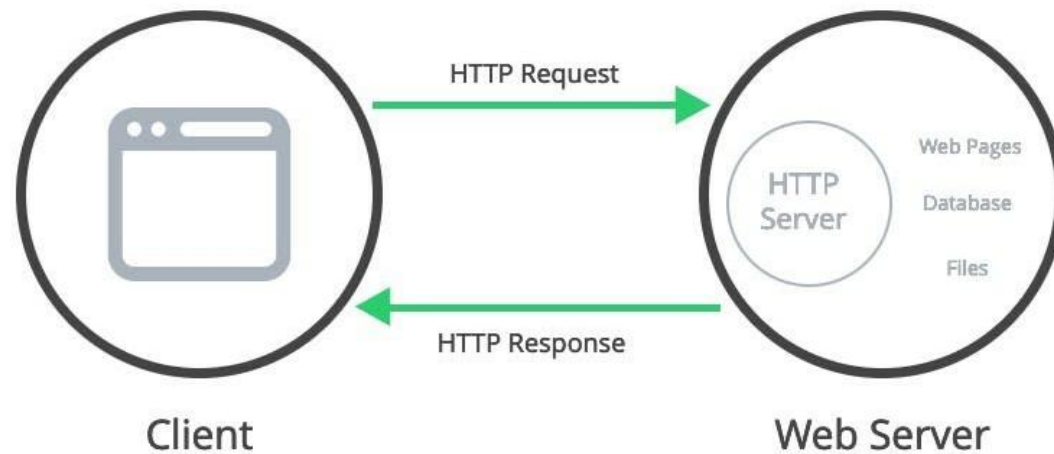
Introdução

- Um protocolo defini **2 regras** principais:
 - **Ordem** de comunicação



Introdução

- Por que **não estudamos** protocolo HTTP em desenvolvimento de aplicações?
 - **Frameworks** de desenvolvimento cuida do trabalho sujo (Node.js)



Introdução

- Porém, no desenvolvimento, lidamos com **recursos HTTP** como:
 - Código de retorno
 - Métodos HTTP
 - Cache
 - Sessões
 - Segurança
 - Etc.

Introdução

- Alguns destes recursos são gerenciados por biblioteca que usamos:
 - **Sessões:** bibliotecas de autenticação e autorização
 - **Segurança:** bibliotecas de segurança
 - Etc.
- Outros recursos podemos utilizar **diretamente** nas aplicações

Introdução

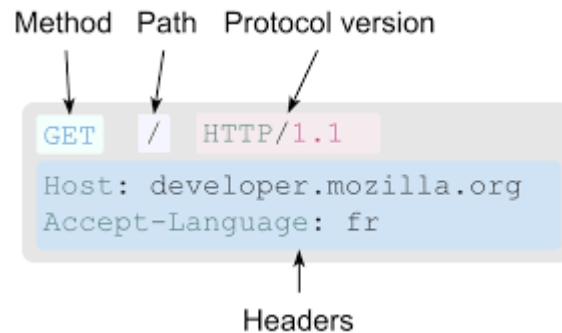
- Neste momento, veremos **alguns** recursos
- Quando estudarmos Serviços Web e Segurança, **veremos mais** recursos
- Vamos utilizar a aplicação de **Gerenciamento de Demandas de TI** nos exemplos

Sumário

- Recursos HTTP
 - Introdução
 - **Métodos**
 - Códigos

Métodos

- O protocolo HTTP é baseado em **métodos**, cada um representa uma **operação**
 - Cada **requisição** de um cliente a um servidor **contém** o método



Métodos

- Métodos HTTP

Método	Função	Características	Exemplo
GET	Buscar/obter dados	Não altera dados Parâmetros na URL	Listar clientes do CRM
POST	Criar novos recursos	Cria novos dados Dados no corpo da requisição	Cadastrar novo lead
PUT	Atualizar completamente	Substitui recurso inteiro Todos os campos necessários	Atualizar perfil completo do cliente
PATCH	Atualizar parcialmente	Modifica campos específicos Mais eficiente para pequenas mudanças	Alterar apenas status do pedido
DELETE	Remover dados	Remove recursos Geralmente irreversível	Cancelar agendamento

Métodos

- Nas **rotas** de uma aplicação, precisamos definir qual **método** será utilizado
 - Rotas da **aplicação** de Gerenciamento de Demandas de TI

```
var express = require('express');
var router = express.Router();
var controllerDemandas = require('../controller/controllerDemandas.js')

// Rota para método GET da criação de demanda
router.get('/cria', controllerDemandas.cria_get);
// Rota para método POST da criação de demanda
router.post('/cria', controllerDemandas.cria_post);
// Rota para consulta de demanda
router.get('/consulta/:id', controllerDemandas.consulta);
// Rota para alteração de status da demanda
router.post('/status/:id/:novo_status', controllerDemandas.altera_status);

module.exports = router;
```

Métodos

- Quando estudarmos Serviços Web, veremos outros métodos
 - Serviços web utilizam os **métodos** para realizar **operações**
 - Aplicações web geralmente se **limitam** a:
 - **GET** para obter dados
 - **POST** para criar, alterar ou excluir dados

Sumário

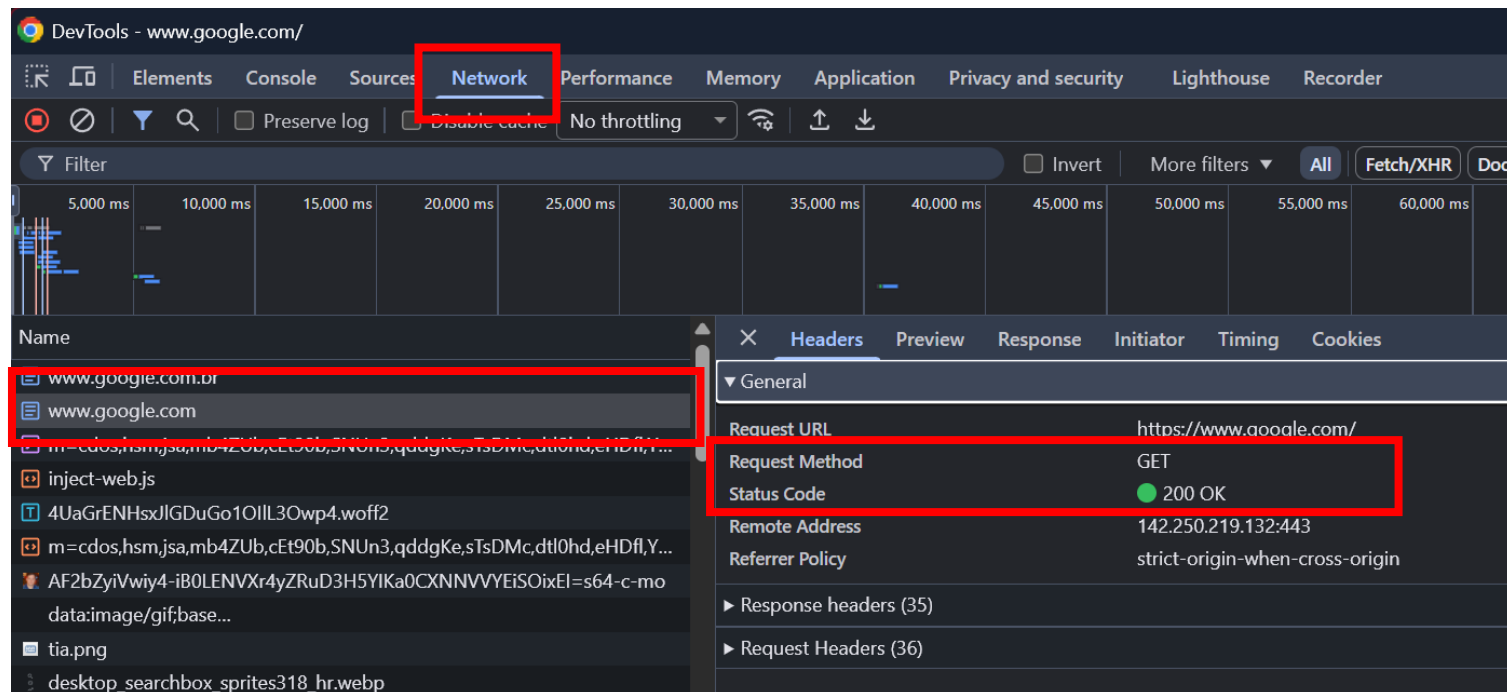
- Recursos HTTP
 - Introdução
 - Métodos
 - **Códigos**

Códigos HTTP

- O protocolo **HTTP** utiliza códigos para:
 - Comunicar o **resultado** de operações para o cliente
 - Informar quando houve **erro**
 - **Classificar** o erro
- Os códigos são enviados nas **respostas** geradas pela aplicação
- Utilizando **frameworks** como Node.js/Express, muitas vezes **não vemos** estes códigos

Códigos HTTP

- Acessando a "**Ferramenta do Desenvolvedor**" ou "**DevTools**" dos navegadores, é possível visualizar estes códigos
 - Aba "Redes" ou "Network"



Códigos HTTP

- Códigos **Oficiais** do protocolo



Códigos HTTP

- Principais códigos utilizados

Level 200

200: OK
201: Created
202: Accepted
203: Non-Authoritative
Information
204: No content

Level 400

400: Bad Request
401: Unauthorized
403: Forbidden
404: Not Found
409: Conflict

Level 500

500: Internal Server Error
501: Not Implemented
502: Bad Gateway
503: Service Unavailable
504: Gateway Timeout
599: Network Timeout

Códigos HTTP

- Principais códigos utilizados



404. That's an error.

The requested URL `/realtime` was not found on this server.
That's all we know.



500. That's an error.

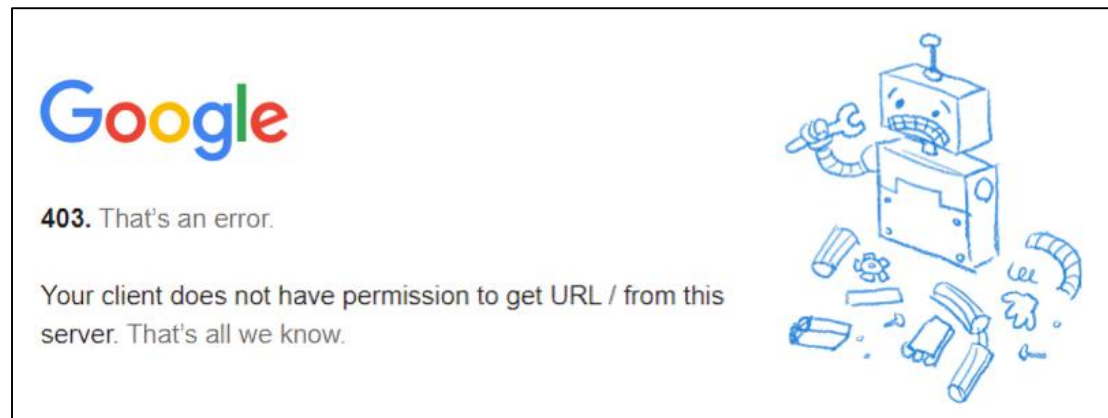
The server encountered an error and could not complete your request.

If the problem persists, please [report](#) your problem and mention this error message and the query that caused it.
That's all we know.



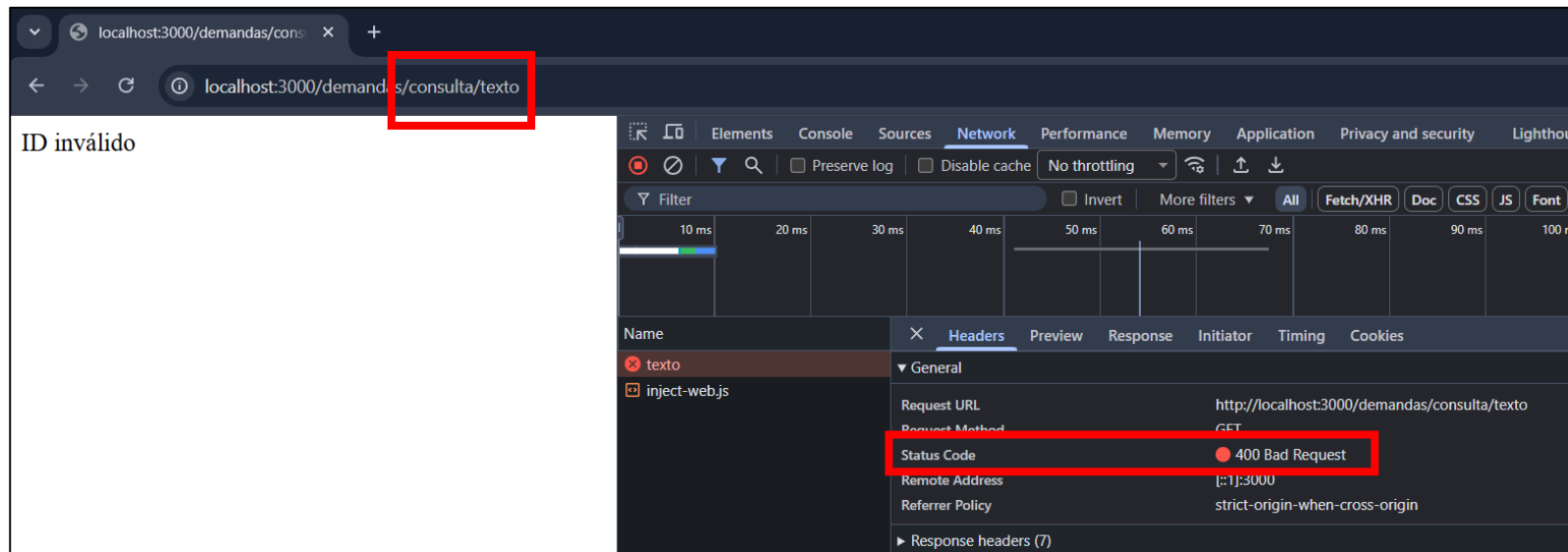
Códigos HTTP

- Principais códigos utilizados



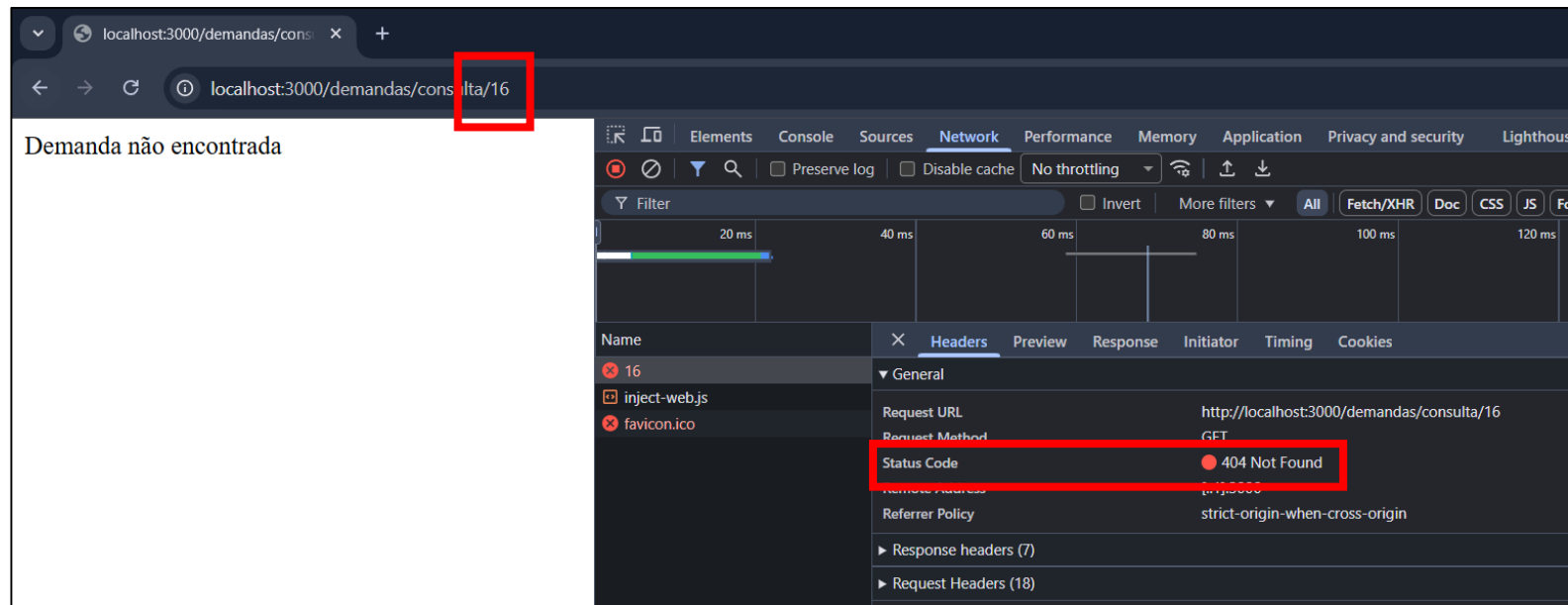
Códigos HTTP

- A aplicação de Gerenciamento de Demandas de TI **também** utiliza estes códigos
 - **Consulta** de demanda com ID incorreto



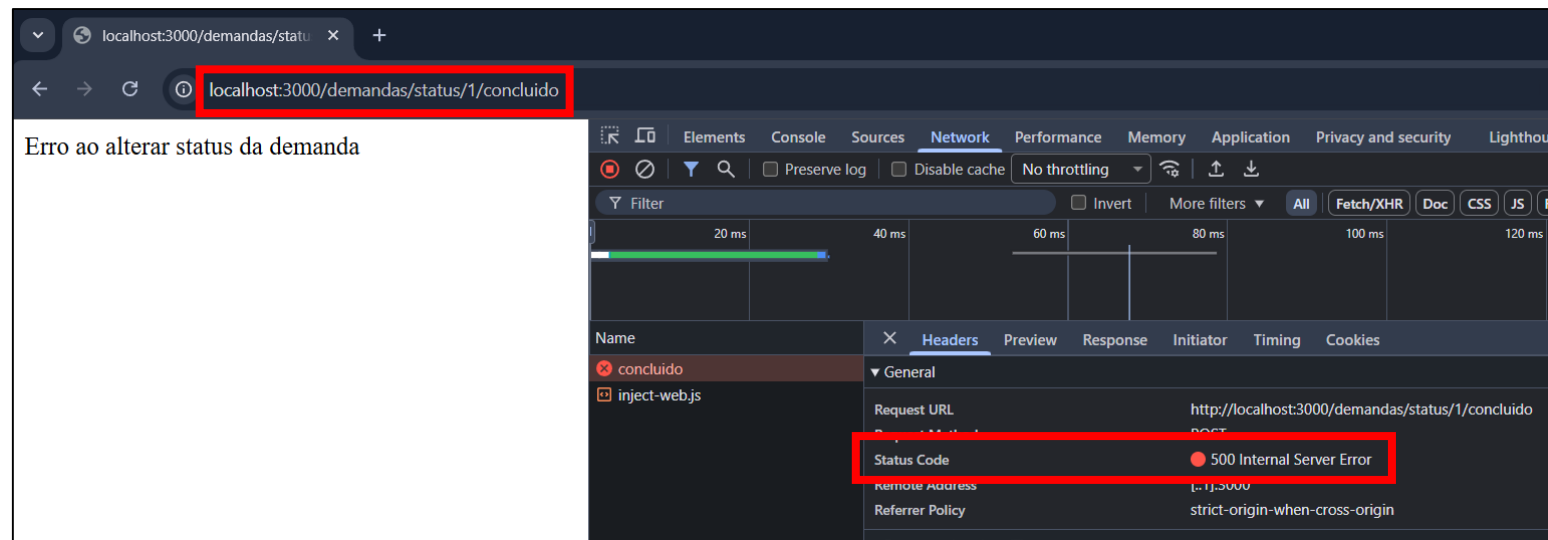
Códigos HTTP

- A aplicação de Gerenciamento de Demandas de TI **também** utiliza estes códigos
 - **Consulta** de demanda que não existe



Códigos HTTP

- A aplicação de Gerenciamento de Demandas de TI **também** utiliza estes códigos
 - **Erro interno**



Códigos HTTP

- Programamos estas respostas com as **melhorias implementadas**
 - **Controller** consulta de demandas:

```
exports.consulta = async function (req, res) {  
  const id_demanda = Number(req.params.id);  
  
  // validação do parâmetro id (deve ser inteiro positivo)  
  if (!id_demanda || Number.isNaN(id_demanda) || !Number.isInteger(id_demanda) || id_demanda <= 0) {  
    return res.status(400).send('ID inválido');  
  }  
  
  try {
```

- Além de **outros** locais/controllers

Códigos HTTP

- Então, é papel do **desenvolvedor selecionar** o código correto para cada resposta
 - O framework **já faz** o básico (páginas que não existe, código de sucesso, etc.)
 - Trecho do apps.js retornando **404** no caso de **rotas incorretas**

```
...  
app.use('/', indexRouter);  
app.use('/demandas', demandasRouter);  
  
// catch 404 and forward to error handler  
app.use(function(req, res, next) {  
  next(createError(404));  
});  
...
```

Códigos HTTP

- O desenvolvedor precisa atuar nos **casos específicos**
 - Erros internos
 - Requisições fora do padrão
 - Acesso não autorizado
 - Etc.
- Veremos mais exemplos destes códigos no **controle de usuário e serviços web**

Exercício

- **Conceitos:** O exercício irá envolver os seguintes **conceitos**:
 - Conceitos vistos em Desenvolvimento Backend I
 - Tratamento de erros com blocos try/catch
 - Validação de dados de parâmetros URL
 - Validação de dados de formulários HTML
 - Configuração correta de códigos de resposta HTTP

Exercício

- **O Problema**

A biblioteca comunitária do bairro precisa digitalizar o controle de acervo para evitar perdas e controlar empréstimos. Seu objetivo é construir o módulo central desta aplicação.

- **Sugestão de Pasta**

- Exercicios > 1.CodigosHTTP

Exercício

- **Definições Gerais**

- Uso de MySQL e Sequelize para gerenciamento de dados
- Utilização de Bootstrap, versão offline
- Utilização opcional de Bootstrap Icons
- Utilização de nodemon para execução do projeto
- Planejamento das rotas e arquivos de rotas é parte do exercício e deve seguir os padrões adotados na disciplina

Exercício

- **Modelos de Dados**
 - Base de Dados: biblioteca
 - Usuário no Banco de Dados: mesmo utilizado nos projetos anteriores
 - Todos os campos como obrigatórios, inclusive do relacionamento entre tabelas

Exercício

- **Modelos de Dados**
 - Tabela: Categoria
 - id
 - nome
 - Tabela: Livro
 - id
 - titulo
 - autor
 - categoria_id (chave estrangeira)
 - status (disponível, emprestado), valor padrão: disponível

Exercício

- **Modelos de Dados**

- Dica:

- Na associação entre Livros e Categoria, adicione um nome para a associação, como abaixo:

```
Livro.belongsTo(Categoria, {  
  as: 'categoria', // nome do atributo de associação  
  ...  
});
```

- Este nome será importante à frente

Exercício

- **Casos de Uso**
 - Listagem de livros (tela principal)
 - Consulta a detalhes de um livro
 - Cadastro de novos livros
 - Alteração de status de um livro
 - Filtro de livros por categoria

Exercício

- **Listagem de Livros (tela principal)**
 - Componentes da Tela Principal
 - Além da listagem de livros, a tela principal deve ter um menu superior
 - Menu superior com o nome do sistema e botão para cadastro de novos livros
 - Menu criado com Navbar do Bootstrap
 - Este menu deve aparecer em todas as páginas, incluído como um "parcial" Bootstrap

Exercício

- **Listagem de Livros (tela principal)**
 - Lista completa de livros do acervo contendo: título, categoria e status
 - Lista de livros deve ser implementada em uma tabela simples do Bootstrap
 - Última coluna da tabela deve ser para ações
 - Alteração de status do livro (botão simples com revezamento entre status (Disponível ou Emprestado))
 - Caso não exista livros no acervo, deve ser exibida mensagem, ao invés da tabela vazia

Exercício

- **Listagem de Livros (tela principal)**
 - No controller da tela principal, caso não exista nenhuma categoria criada ainda, devem ser criadas, pelo menos, 4 categorias no banco de dados
 - Você deve implementar a verificação e cadastro
 - Categorias não terá telas de CRUD para simplificar o exercício
 - A função "bulkCreate" do Sequelize pode ser utilizada para criação de múltiplos registros ao mesmo tempo

Exercício

- **Listagem de Livros (tela principal)**

- Dica:

- Na consulta da lista de livros, adicione o modelo de dados de Categoria, utilizando o nome dado ao relacionamento no arquivo de modelo de dados

```
const livros = await Livro.findAll({ include: [{ model: Categoria, as: 'categoria' }] });
```

- A consulta garante que cada objetivo "livro", terá um atributo "categoria" com as informações da categoria do livro

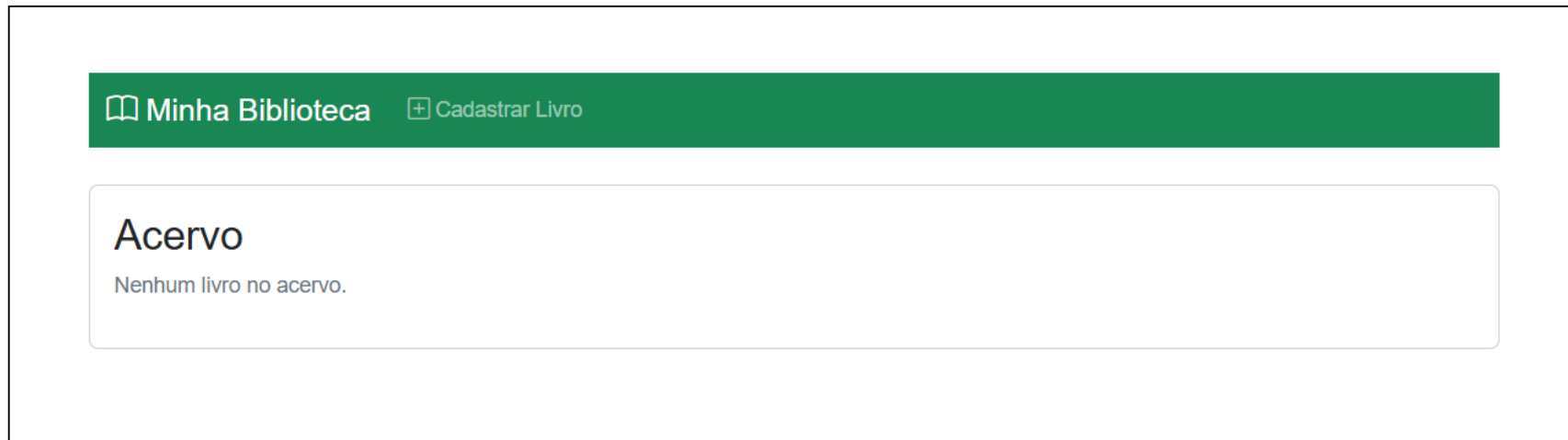
Exercício

- Listagem de Livros (tela principal)

Minha Biblioteca + Cadastrar Livro		
Acervo		
Título	Categoria	Ações
Clean Code	Informática	Disponível

Exercício

- Listagem de Livros (tela principal)



Exercício

- **Consulta de Livros**

- Acesso clicando no título do livro, na tela principal
- Deve ser uma tela com todas as informações sobre o livro, incluindo a categoria
- Botão voltar, para a tela principal
- Uso de Card simples do Bootstrap
- Formatação para as datas de criação e atualização
- Uso de badges do Bootstrap para status do livro

Exercício

- Consulta de Livros

Minha Biblioteca		
Acervo		
Título	Categoria	Ações
Clean Code	Informática	disponível
Dominando o Node.js	Informática	emprestado

Exercício

- Consulta de Livros

📖 Minha Biblioteca

☰

Detalhes do Livro

Título: Clean Code

Autor: Robert C. Martin

Categoria: Informática

Status: disponível

Data de Criação: 07/04/2026

Data de Atualização: 07/04/2026

Voltar

Exercício

- Consulta de Livros

📖 Minha Biblioteca

Detalhes do Livro

Título: Dominando o Node.js

Autor: Ulises Gascón

Categoria: Informática

Status: emprestado

Data de Criação: 07/04/2026

Data de Atualização: 07/04/2026

Voltar

Exercício

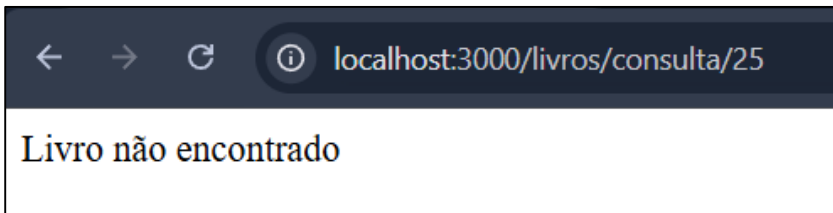
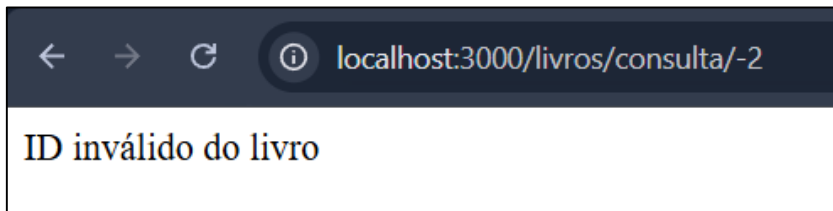
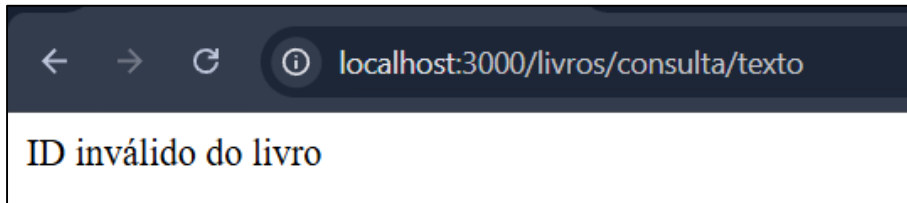
- **Consulta de Livros**

- Validações

- ID passado como parâmetro deve ser **obrigatório**, caso contrário retorna mensagem de erro, com código HTTP adequado
 - ID passado como parâmetro deve ser **número inteiro**, caso contrário retorna mensagem de erro, com código HTTP adequado
 - ID passado como parâmetro deve ser **positivo**, caso contrário retorna mensagem de erro, com código HTTP adequado
 - ID passado como parâmetro deve ser **de um livro cadastrado**, caso contrário retorna mensagem de erro, com código HTTP adequado
 - Erro ao consultar registro no Banco de Dados

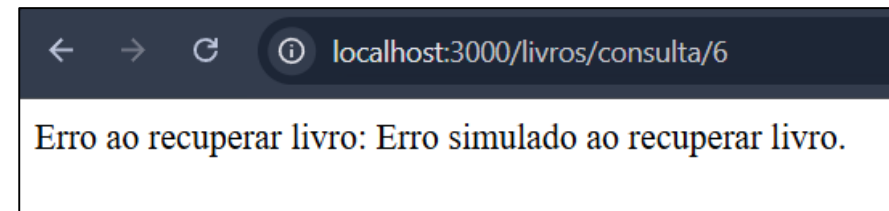
Exercício

- Consulta de Livros



Simulação de erro ao consultar registro com:

```
// erro simulado para teste  
throw new Error('Erro simulado ao recuperar livro.');
```



Exercício

- **Cadastro de Novos Livros**

- Formulário para cadastro de novos livros, incluindo indicação da categoria
- O cadastro é acessado do item de menu, da barra de menu superior
- As categorias dos livros são cadastradas no controller da página principal, para não desviar o foco do exercício
- Após o cadastro de um novo livro, o usuário deve ser redirecionado para a tela principal

Exercício

- Cadastro de Novos Livros

 Minha Biblioteca  Cadastrar Livro

Novo Livro

Título

Autor

Categoria

Cadastrar Livro

Exercício

- Cadastro de Novos Livros

Minha Biblioteca + Cadastrar Livro		
Acervo		
Título	Categoria	Ações
Clean Code	Informática	disponível
Dominando o Node.js	Informática	emprestado
A Arte da Guerra	Ficção	disponível

Exercício

- **Cadastro de Novos Livros**

- Validações

- Todos os campos obrigatórios (conferência no controller)
 - Categoria deve ser número, inteiro, positivo
 - Em caso de erro, usuário deve ser redirecionado para a própria tela de cadastro
 - Controller deve verificar erro de criação do registro em Banco de Dados também
 - Tela de cadastro com opção para exibir mensagens de erro
 - Configuração dos códigos HTTP corretos para cada resposta
 - Para testar as validações, basta remover temporariamente os atributos "required" dos campos do formulário de cadastro

Exercício

- Cadastro de Novos Livros

 Minha Biblioteca  Cadastrar Livro

Novo Livro

- Título é obrigatório

Título

Autor

Stephen Hawking

Categoria

Ficção

Cadastrar Livro

Exercício

- Cadastro de Novos Livros

 Minha Biblioteca  Cadastrar Livro

Novo Livro

- Autor é obrigatório

Título

Uma breve história do tempo

Autor

Categoria

Ficção

Cadastrar Livro

Exercício

- Cadastro de Novos Livros

📖 Minha Biblioteca

+ Cadastrar Livro

Novo Livro

• Categoria é obrigatória

Título

Uma breve história do tempo

Autor

Stephen Hawking

Categoria

Selecione a categoria ▼

Cadastrar Livro

Exercício

- Cadastro de Novos Livros

 Minha Biblioteca  Cadastrar Livro

Novo Livro

• Categoria inválida

Título

Uma breve história do tempo

Autor

Stephen Hawking

Categoria

Selecione a categoria ▼

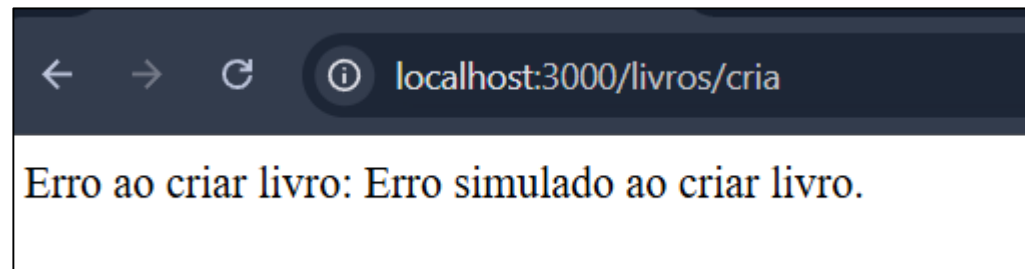
Cadastrar Livro

Para categorias que não são:

- número
- inteiro
- positivo

Exercício

- Cadastro de Novos Livros



Simulação de erro ao criar registro com:

```
//disparar erro para teste  
throw new Error('Erro simulado ao criar livro.');
```

Exercício

- **Alteração de Status do Livro**

- A alteração do status será feita por meio do botão próprio da tela principal
- Ao clicar no botão, o status do livro deve ser revezado entre "Disponível" e "Emprestado"
- Após a alteração do status, o usuário deve ser redirecionado para a tela principal novamente

Exercício

- Alteração de Status do Livro

Acervo		
Título	Categoria	Ações
Clean Code	Informática	↻ disponível
Dominando o Node.js	Informática	↻ emprestado
A Arte da Guerra	Ficção	↻ disponível

Acervo		
Título	Categoria	Ações
Clean Code	Informática	↻ emprestado
Dominando o Node.js	Informática	↻ emprestado
A Arte da Guerra	Ficção	↻ disponível

Exercício

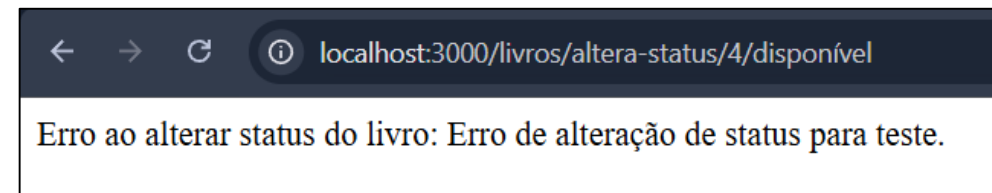
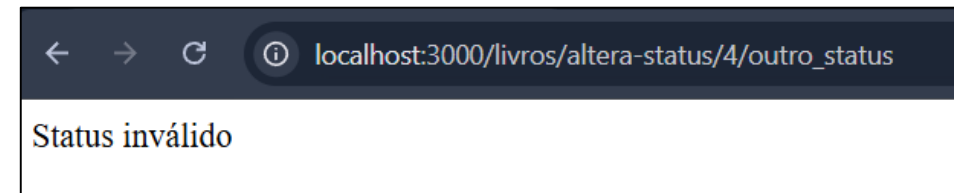
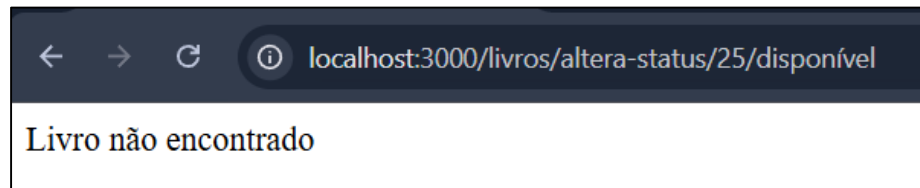
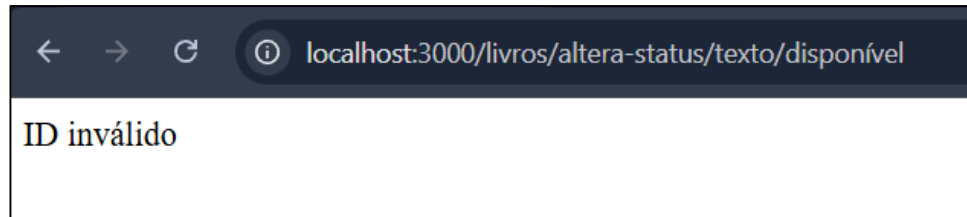
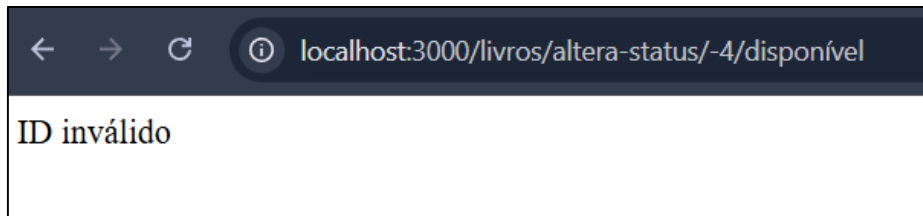
- **Alteração de Status do Livro**

- Validações

- ID passado como parâmetro na URL deve ser número
 - ID passado como parâmetro na URL deve ser inteiro
 - ID passado como parâmetro na URL deve ser positivo
 - ID passado como parâmetro na URL deve de um livro cadastrado no banco de dados
 - Novo status deve ser um dos status válidos ('disponível', 'emprestado')
 - Controller deve verificar erro de alteração do registro em Banco de Dados também

Exercício

- Alteração de Status do Livro



A maioria destes erros podem ser induzido alterando o campo 'action' do formulário do DevTools do navegador web

Exercício

- **Filtro por Categoria**

- Na tela principal, haverá local para carregar um botão para cada categoria cadastrada no banco de dados
 - Preferencialmente acima da tabela com a lista de livros
- Quando o usuário clicar sobre um botão de categoria, os livros exibidos na tela principal devem ser filtrados de acordo com a categoria selecionada
 - Ao fazer um filtro, os botões de categoria deve ser substituídos por botão para cancelar o filtro (exibindo todos os registros novamente)
 - Também deve ser exibida a categoria do filtro atual

Exercício

- **Filtro por Categoria**

- Como resultado de todas as operações de filtro e cancelamento de filtro, o usuário deve sempre ser redirecionado para a tela principal
- Dica:
 - O filtro pode ser repassado à rota da tela principal por meio de 'query' na URL
 - `"/?categoria=Informática"`
 - No controller, é possível obter uma query desta forma:

```
const id_categoria_filtro = req.query.categoria
```

Exercício

- **Filtro por Categoria**

- Dica:

- Cada botão de filtro, deve estar configurado com a URL e query correta
 - No controller, deve ser verificado se há ou não uma query na URL
 - Assim será possível saber se deve ser carregada a lista de livros com ou sem filtro de categoria

Exercício

- Filtro por Categoria

📖 Minha Biblioteca

+ Cadastrar Livro

🔍 Informática

🔍 Ficção

🔍 Autoajuda

🔍 Ciência

Acervo

Título	Categoria	Ações
Clean Code	Informática	🔄 emprestado
Dominando o Node.js	Informática	🔄 emprestado
A Arte da Guerra	Ficção	🔄 disponível

Exercício

- Filtro por Categoria

📖 Minha Biblioteca

+ Cadastrar Livro

Filtrando por: Informática

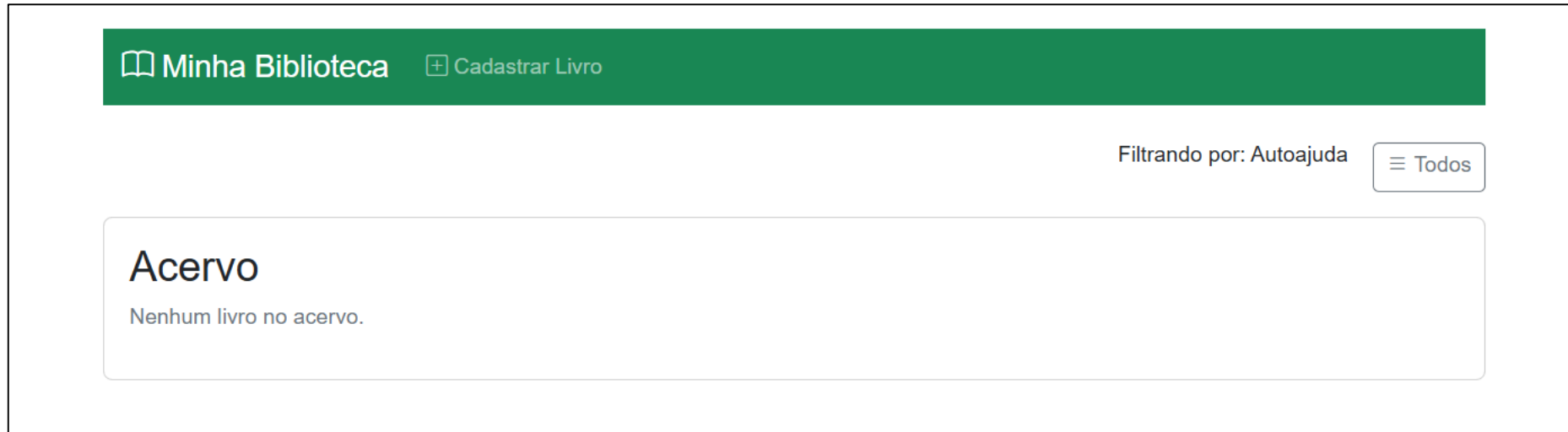
Todos

Acervo

Título	Categoria	Ações
Clean Code	Informática	🔄 emprestado
Dominando o Node.js	Informática	🔄 emprestado

Exercício

- **Filtro por Categoria**



Exercício

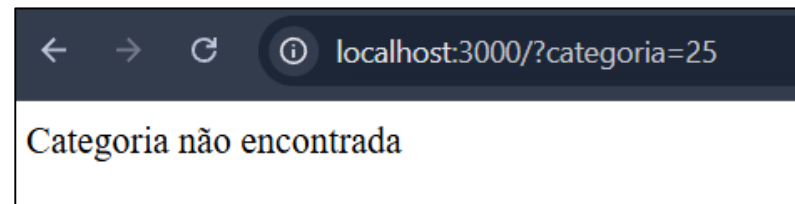
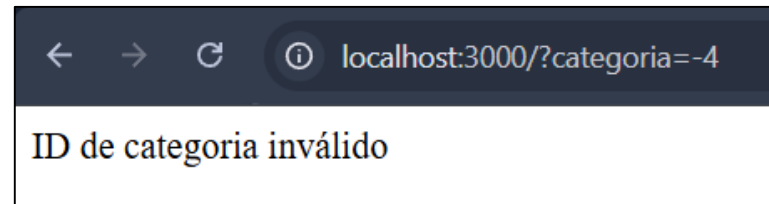
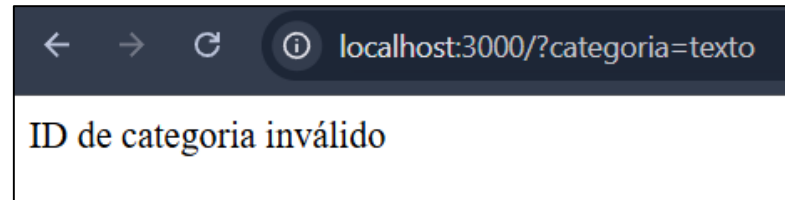
- **Filtro por Categoria**

- Validações

- ID da categoria passada via query na URL deve ser número
 - ID da categoria passada via query na URL deve ser inteiro
 - ID da categoria passada via query na URL deve ser positivo
 - ID da categoria passada via query na URL deve ser de uma categoria cadastrada no Banco de Dados

Exercício

- **Filtro por Categoria**



Sumário

- Recursos HTTP
 - Introdução
 - Métodos
 - Códigos